Hackathon Prep

Master Docker & GPU Deployments

A core requirement for your final submission will be **containerizing your solution using Docker** so it can run seamlessly on our GPU-enabled evaluation servers.

If you are new to Docker or haven't deployed machine learning applications inside containers before, this guide is for you. Please read through and practice before the event begins!

Why Docker?

Docker allows you to package your application, dependencies, and environment configurations into a single, portable unit called a **container**. This ensures that if your code works on your laptop, it will work exactly the same way on our grading servers.

Learning Resources & References

Rather than giving you code templates that might not fit your specific architecture, we have curated the best resources for you to learn how to containerize Python APIs and ML models from scratch.

1. Video Tutorial — The Absolute Basics of Docker

If you have never written a Dockerfile before, start here. This step-by-step, hands-on tutorial walks you through creating a Dockerfile, building an image, and running a container from the ground up:

📺 Dockerfile > Docker Image > Docker Container | Beginners Hands-On | Step by Step
Automation Step by Step — available on YouTube

2. Official Documentation — Containerizing Python APIs

Most participants use frameworks like FastAPI to build their API endpoints. Here are the best official guides to containerizing your Python web servers:

📖 FastAPI in Containers - Docker (Official FastAPI Guide)
`https://fastapi.tiangolo.com/deployment/docker/`

📖 Build your Python image (Official Docker Documentation)
`https://docs.docker.com/language/python/build-images/`

3. Handling GPUs inside Docker

Since you will be working with AI models that require GPU acceleration for fast inference, you will need to know how to pass your host machine's GPU into your Docker container. Familiarize yourself with this toolkit:

📖 NVIDIA Container Toolkit Installation Guide
`https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/install-guide.html`

> ✅ Once the NVIDIA Container Toolkit is installed, you can pass your GPU into any container with the --gpus all flag:
> `docker run --gpus all -p 8000:8000 your-image:latest`

A Crucial Heads-Up on Deployment Times

When working with AI and Large Language Models, your Docker images will inevitably be very large — often **several gigabytes** in size. Please keep the following in mind:

- Building and testing these images locally takes time.
- Uploading (pushing) your final image to a registry like Docker Hub will be highly dependent on your internet speed and can take a significant amount of time.

🚨 Do NOT leave your Docker build and upload until the final hour of the hackathon, or you risk missing the submission deadline. We highly recommend practicing this end-to-end process beforehand so you are fully prepared.

Handling Hugging Face Tokens Securely

You will likely need to download gated models (like Llama) inside your container. **Never hardcode your Hugging Face token** in your code or Dockerfile.

Instead, your code should read the token from an environment variable:

```
import os
hf_token = os.getenv("HF_TOKEN")
```

Then pass the token at runtime when starting your container:

```
docker run --gpus all -p 8000:8000 -e HF_TOKEN=your_token_here
your-image:latest
```

⚠ Never commit your HF token to source code or include it in your ZIP / Docker image. Use environment variables exclusively.